Database Users, Roles, Privileges, and Access Control

Group 7 (CAMPUSAFE)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** CAMPUSAFE **Repository Link:** **https://github.com/jeffersonorpesa2-ops/grp07-campussafe** **Our Group Repository:**
Projects set up	**The CampusSafe repository was used as the basis for implementing the database access-control plan. The project is a campus safety management application designed to support campus safety-related operations and manage information within its database.** **The first step was to examine the existing project files and understand how the application connects to its database. The README file, database configuration, and existing SQL files were reviewed to identify the database name, tables, and other database objects that needed access restrictions.** **The database used for this activity was named [INSERT ACTUAL DATABASE NAME]. The existing database structure was retained, and the access-control implementation was prepared separately to avoid modifying the original system functionality.** **The existing database tables, sequences, and other database objects should be identified from the actual CampusSafe database structure before documenting their names and purposes.** **After reviewing the project files and database requirements, a separate folder named Activity-4 was prepared to organize the SQL scripts and access-control documentation.** **The following files were created:** ● **01_create_roles.sql** ● **02_grant_privileges.sql** ● **03_revoke_privileges.sql** ● **04_test_privileges.sql** ● **access_control_plan.md**

	These files were organized to make the implementation easier to understand, execute, and maintain.
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each role. The implementation follows the Principle of Least Privilege, meaning that each role should only receive the permissions necessary for its responsibilities.** **Three PostgreSQL group roles were established.** **Administrator Role (campussafe_admin_role)** **The administrator role is responsible for managing and maintaining the CampusSafe database. It receives administrative privileges on the existing tables and sequences within the public schema.** **Its responsibilities include viewing database records, inserting new records, updating existing information, deleting records when necessary, and managing database objects.** **This role is intended only for authorized database administrators and should not be used for ordinary application operations.** **Application Role (campussafe_app_role)** **The application role is intended for the CampusSafe application that interacts with the database. It allows the application to perform normal database operations.** **Its responsibilities include retrieving information needed by the application, adding new records, updating existing information, and deleting records when required by application functionality.** **The application role receives SELECT, INSERT, UPDATE, and DELETE privileges on the existing tables in the public schema, together with the necessary sequence permissions.** **It is restricted from creating database objects to help prevent unauthorized changes to the database structure.** **Auditor Role (campussafe_auditor_role)** **The auditor role is intended for authorized users who only need to view database information. It receives SELECT privileges on the existing**

	tables in the public schema. **Its responsibilities include viewing records, inspecting database information, and reviewing available information for auditing and reporting purposes.** **The auditor role cannot insert, update, or delete records. This ensures that users assigned to this role can inspect information without making unauthorized modifications.** **Access-Control Matrix** **Database Operations** / **Administrator** / **Application** / **Auditor** **CONNECT** / **Allowed** / **Allowed** / **Allowed** **SELECT** / **Allowed** / **Allowed** / **Allowed** **INSERT** / **Allowed** / **Allowed** / **Denied** **UPDATE** / **Allowed** / **Allowed** / **Denied** **DELETE** / **Allowed** / **Allowed** / **Denied** **CREATE** / **Allowed** / **Denied** / **Denied** **Sequence USAGE** / **Allowed** / **Allowed** / **Denied** **The permissions listed above represent the planned access-control configuration. Actual permissions must be verified against the**

	implemented SQL scripts and the application's database requirements. **Database Users and Role Assignments** **The following database login accounts were prepared:** **Database User — Assigned Role** **campussafe_admin — campussafe_admin_role** **campussafe_app — campussafe_app_role** **campussafe_auditor — campussafe_auditor_role** **The three group roles were created with the NOLOGIN attribute. This means they function as permission groups and must be assigned to actual login accounts before individual users or application accounts can use their permissions.** **The login accounts were created using the LOGIN attribute and assigned to their corresponding group roles through PostgreSQL GRANT statements.**
SQL scripts summary	**Four main SQL implementation and verification scripts were prepared to implement and check the access-control configuration.** **File 1: 01_create_roles.sql** **The first SQL script was used to create the three PostgreSQL group roles and their corresponding login accounts.** **The following group roles were created:**

	• **campussafe_admin_role** • **campussafe_app_role** • **campussafe_auditor_role** **The following database login accounts were also created:** • **campussafe_admin** • **campussafe_app** • **campussafe_auditor** **Each login account was assigned to its corresponding group role using PostgreSQL GRANT statements.** **This allows the database users to inherit the permissions assigned to their respective roles.** **The administrator account is intended for database administration, the application account is intended for normal system operations, and the auditor account is intended for read-only access.** **File 2: 02_grant_privileges.sql** **The second SQL script was used to assign the required database permissions to each role.** **First, the three group roles were given CONNECT permission on the CampusSafe database.** **The administrator, application, and auditor roles were also given USAGE permission on the public schema so they could access the database objects for which they have appropriate privileges.** **The administrator role received ALL PRIVILEGES on all existing tables and sequences in the public schema. It was also given CREATE permission on the public schema.** **The application role received SELECT, INSERT, UPDATE, and DELETE permissions on all existing tables in the public schema. It also received the necessary USAGE and SELECT privileges on sequences for database operations.** **The auditor role received SELECT privileges only on all existing tables in the public schema.**

	The use of ON ALL TABLES IN SCHEMA public allows PostgreSQL to assign permissions to all existing tables without manually specifying each table name. **This makes the privilege assignment easier to maintain when working with a database containing multiple tables.** **The actual database name must be used in the GRANT CONNECT statement. The placeholder database name must be replaced with the existing CampusSafe database name to avoid database connection errors.** **File 3: 03_revoke_privileges.sql** **The third SQL script was used to remove privileges that should not be available to restricted roles.** **The auditor role was restricted from performing INSERT, UPDATE, and DELETE operations on database tables.** **The script also ensures that the auditor role retains SELECT permissions so authorized users can continue viewing database information.** **The application and auditor roles were also restricted from creating database objects in the public schema.** **This prevents ordinary application and auditing accounts from making unauthorized structural changes to the database.** **The administrator role retains its administrative permissions, while the application and auditor roles are limited to the operations required by their responsibilities.** **This helps maintain the Principle of Least Privilege by preventing unnecessary access rights.** **File 4: 04_test_privileges.sql** **The fourth SQL script was used to check whether the assigned permissions were correct.** **The script contains queries that display the created roles, list their table privileges, and test specific permissions using PostgreSQL's**

	has_table_privilege() function. **The verification focused on checking whether:** ● **The administrator can perform SELECT operations.** ● **The administrator can perform INSERT operations.** ● **The administrator can perform UPDATE operations.** ● **The administrator can perform DELETE operations.** ● **The application role can view records.** ● **The application role can insert records.** ● **The application role can update records.** ● **The application role can delete records.** ● **The auditor role can select records.** ● **The auditor role cannot insert records.** ● **The auditor role cannot update records.** ● **The auditor role cannot delete records.** **These checks help confirm that each role follows the intended access-control plan.** **The verification queries must use actual CampusSafe table names. Permission checks should also be performed using separate login accounts to confirm that the restrictions work as intended.**
Verification results	**After preparing the SQL scripts, they were executed in the PostgreSQL Query Tool through pgAdmin 4.** **The scripts were executed in the correct order to avoid permission-related errors.** **First Verification: Role Creation** **The first script was executed to create the three group roles and their corresponding login accounts.** **A verification query was used to display the created roles from the PostgreSQL system catalog.** **Expected result:** **All three group roles and their corresponding login accounts should appear in the query results.**

	Second Verification: Privilege Assignment **The second script was executed to assign the required database, schema, table, and sequence privileges.** **The execution messages were checked to determine whether the commands completed successfully.** **Expected result:** **Each role should receive the permissions defined in the access-control plan.** **The administrator should receive administrative privileges, the application should receive the required application privileges, and the auditor should receive read-only privileges.** **Third Verification: Privilege Restrictions** **The third script was executed to remove unnecessary privileges from restricted roles.** **Expected result:** ● **The auditor role should not have INSERT privileges.** ● **The auditor role should not have UPDATE privileges.** ● **The auditor role should not have DELETE privileges.** ● **The application role should retain its required application privileges.** ● **The auditor role should retain SELECT privileges.** ● **The application and auditor roles should not have CREATE privileges on the public schema.** **Fourth Verification: Permission Checking** **The fourth script was executed to verify the permissions using PostgreSQL queries.** **The expected results were:** **Verification** **Expected Result**

	Administrator can SELECT records	**TRUE**
	Administrator can INSERT records	**TRUE**
	Administrator can UPDATE records	**TRUE**
	Administrator can DELETE records	**TRUE**
	Application can SELECT records	**TRUE**
	Application can INSERT records	**TRUE**
	Application can UPDATE records	**TRUE**
	Application can DELETE records	**TRUE**
	Auditor can SELECT records	**TRUE**
	Auditor can INSERT records	**FALSE**

	Auditor can UPDATE records FALSE **Auditor can DELETE records FALSE** **The expected results demonstrate that the administrator has the intended database management permissions, the application role can perform normal application operations, and the auditor role is restricted to viewing records.** **Note: These are expected results, not confirmed execution results. Actual results must be recorded after running the scripts and testing the permissions using the assigned PostgreSQL roles.** --- ## Screenshots **The following screenshots should be included as supporting evidence of the implementation and verification process:** **Screenshot 1: G7_create_roles.png** **Screenshot 2: G7_grant_privileges.png** **Screenshot 3: G7_revoke_privileges.png** **Screenshot 4: G7_test_privileges.png** **The screenshots serve as supporting evidence of the SQL execution process, role creation, privilege assignment, privilege restrictions, and permission verification.** **Additional screenshots of the CampusSafe database structure and database setup may also be included to demonstrate that the existing database was successfully initialized.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, the following possible issues were considered and addressed.**

	Database Name Mismatch **One of the errors encountered during the preparation of the grant privileges script was:** **ERROR: database "campussafe_db" does not exist.** **This happened because campussafe_db was used as a placeholder database name and did not match the actual PostgreSQL database name.** **To resolve the issue, the available databases were checked using the following query:** **SELECT datname FROM pg_database WHERE datistemplate = false;** **The actual CampusSafe database name must be used in the GRANT CONNECT statement to ensure that the privileges are assigned to the correct database.** **Existing Database Roles** **Running role creation commands repeatedly may cause errors if the roles already exist.** **To prevent this issue, the role creation script should check whether each role exists before creating it.** **This helps avoid duplicate role creation errors when executing the SQL scripts multiple times.** **Incorrect Execution Order** **Granting privileges before creating the required roles can result in errors.** **To prevent this issue, the scripts were organized and executed in the following order:** **1. 01_create_roles.sql 2. 02_grant_privileges.sql 3. 03_revoke_privileges.sql 4. 04_test_privileges.sql** **This ensures that the required roles exist before permissions are**

	assigned and tested. **Insufficient Database Permissions** **Some PostgreSQL commands require appropriate administrative privileges.** **The scripts should be executed using an authorized PostgreSQL administrator account to ensure that the role creation and privilege assignment commands can be completed successfully.** **Incorrect Table Names** **Granting privileges to tables that do not exist can result in relation errors.** **To avoid this issue, the privilege assignment script uses:** **ON ALL TABLES IN SCHEMA public** **This allows PostgreSQL to apply the specified privileges to all existing tables in the public schema without manually entering individual table names.** **However, the actual CampusSafe database structure must be checked to confirm that the required tables are located in the public schema.** **Future Database Objects** **Newly created tables and sequences may not automatically receive the same permissions as existing objects.** **Default privileges should be configured for the correct object owner when necessary.** **This helps ensure that future database objects receive the intended access permissions.** **Application and Auditor Permission Restrictions** **The application and auditor roles have different responsibilities.** **The application role is allowed to perform the database operations required by the CampusSafe application.** **The auditor role is restricted to SELECT operations and must not be**

	allowed to insert, update, or delete records. **The verification script checks these permissions to confirm that each account follows the intended access-control configuration.** **Database Connection and Schema Access** **Database-level CONNECT privileges do not automatically provide access to tables.** **The appropriate schema and table privileges must also be granted.** **To address this, the grant privileges script includes CONNECT on the database, USAGE on the public schema, and the required table and sequence privileges.**
Feedback to owners	**The CampusSafe application provides database functionality for managing campus safety-related information and supporting its system operations. Implementing Role-Based Access Control can further improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** **1. Maintain separate database roles for administrators, application access, and auditors.** **2. Avoid using administrator-level database credentials for regular application activities.** **3. Assign actual login accounts to the appropriate group roles.** **4. Use strong passwords and protect database credentials.** **5. Review database permissions whenever new tables or application features are added.** **6. Restrict access to sensitive campus safety information.** **7. Regularly review database roles and privileges to prevent unnecessary access.** **8. Use read-only accounts for users who only need to view database information.** **9. Apply appropriate database privileges to newly created tables and sequences.** **10. Perform regular database backups to help prevent data loss.** **11. Avoid granting unnecessary administrative privileges to application accounts.** **12. Regularly test database access restrictions to ensure that unauthorized operations are prevented.** **These recommendations can help improve the security, reliability, and**

	maintainability of the CampusSafe application.
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation.** **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, prepare the database access-control plan, and identify possible implementation issues and their corresponding solutions.** **The generated SQL commands and documentation must be reviewed, adjusted to match the actual CampusSafe database structure, and validated through PostgreSQL execution using pgAdmin 4.** **The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.** **All passwords and other sensitive credentials must be excluded from the submitted repository, screenshots, and documentation.**